IDS Project — Complete Technical Reference

Mustafa Göçmen | LIMOS-CNRS Internship 2026

PART 1 — ZEEK

What is Zeek?

Zeek (formerly Bro) is a **passive network security monitor**. It sits on a network interface, reads every packet that flows through it, reconstructs network sessions, identifies protocols, and writes structured log files. It does NOT block traffic — it only observes and records.

The key distinction from tools like Suricata: Zeek is not signature-based. It doesn't match packets against a rule database. Instead, it builds a semantic understanding of network activity and lets you write scripts to define what "interesting" means. This makes it extremely flexible for research.

How Zeek Works Internally

1. Packet Capture Layer

Zeek uses **libpcap** to read raw packets from a network interface in promiscuous mode. Promiscuous mode means the interface accepts ALL packets on the network segment, not just those addressed to the machine. This is how passive monitoring works — the traffic doesn't need to go through Zeek, it just needs to be visible on the same wire.

In your setup:

```
Dell (attacker) → Switch → Raspberry Pi (Zeek listening on eth0)
MacBook (benign) → Switch → Raspberry Pi (Zeek listening on eth0)
```

Zeek on the Pi sees everything because the switch broadcasts to all ports (unmanaged switch behavior).

2. Protocol Analysis Engine

Zeek has protocol analyzers for TCP, UDP, HTTP, DNS, SSL/TLS, SSH, FTP, SMTP, and many more. These analyzers work by inspecting packet payloads — they don't rely on port numbers. HTTP on port 8080 is still detected as HTTP. This is called **deep packet inspection** at the semantic level.

3. Event Engine

As Zeek processes packets, it generates **events** — things like `connection_established`, `http_request`, `dns_request`, `ssl_handshake_complete`. Your Zeek scripts can hook into these events to add custom logic.

4. Connection State Machine

Zeek tracks the full lifecycle of every TCP connection through a state machine:

State	Meaning
`S0`	SYN sent, no SYN-ACK received (connection attempt, no reply)
`S1`	SYN + SYN-ACK seen, but no ACK (half-open)
`SF`	Normal full connection: SYN → SYN-ACK → ACK → FIN (successful)
`REJ`	SYN sent, RST received (connection refused)
`S2`	Connection established, originator sent FIN, not acknowledged
`S3`	Connection established, responder sent FIN, not acknowledged
`RSTO`	Connection established, originator sent RST
`RSTR`	Connection established, responder sent RST
`OTH`	No SYN seen, just midstream traffic

For your project:

- Slowloris attack → Zeek logs hundreds of `S0` connections from `192.168.10.2`
- Normal Selenium bot → Zeek logs `SF` connections (complete, successful HTTP)

Zeek Log Files — Detailed

All logs live at: `/opt/zeek/logs/current/`

conn.log

The most important file. Every single TCP/UDP/ICMP flow gets one row.

Key fields (tab-separated):

```
ts          — Unix timestamp (e.g. 1782212684.332354)
uid         — Unique connection identifier (e.g. Cbod3x1qqJZ6BuWlj)
id.orig_h   — Source IP (192.168.10.2)
id.orig_p   — Source port (54309)
id.resp_h   — Destination IP (192.168.10.1)
id.resp_p   — Destination port (80)
proto       — Protocol (tcp/udp/icmp)
service     — Detected application layer protocol (http/dns/ssl/-)
duration    — How long the connection lasted (seconds)
orig_bytes  — Bytes sent by originator
resp_bytes  — Bytes sent by responder
conn_state  — Connection state (S0/SF/REJ/etc.)
orig_pkts   — Packets from originator
resp_pkts   — Packets from responder
```

Reading Slowloris in conn.log:

```
1782212684.33  Cbod3x  192.168.10.2  54309  192.168.10.1  80  tcp  http  0.014
189  0  S0  T  T  0  ScAD  5  401  0  0-
```

- `S0` = connection never completed
- `resp_bytes = 0` = server never got to respond
- `duration = 0.014` = very short (socket opened and Slowloris trickled a partial header)
- `orig_bytes = 189` = small partial HTTP header sent

Reading Selenium bot in conn.log:

```
1782212700.11  CXyz12  192.168.10.3  52100  192.168.10.1  80  tcp  http  0.003
312  854  SF  ...
```

- `SF` = normal complete connection
- `resp_bytes = 854` = server actually sent the HTML response
- Looks completely normal

http.log

Generated for every HTTP transaction.

Key fields:

```
ts          — timestamp
uid         — links to conn.log entry
id.orig_h   — client IP
method      — GET/POST/PUT/DELETE
host        — HTTP Host header value
uri         — requested path (/index.html, /, /about)
referrer    — HTTP Referer header
user_agent  — browser/client identifier string
request_body_len  — request body size
response_body_len — response body size
status_code — HTTP response code (200/404/503/408)
```

Slowloris in http.log:

- Status `408` (Request Timeout) — server waited for complete headers, gave up
- No User-Agent (Slowloris doesn't send one, or sends incomplete headers)

Selenium bot in http.log:

- Status `200` (OK)
- User-Agent: `Mozilla/5.0 (Macintosh; Intel Mac OS X...) Chrome/148...`
- This is the key observation: **indistinguishable from a real human browser**

dns.log

Every DNS query and response.

```
query       — domain being looked up
qtype_name  — query type (A, AAAA, MX, TXT, etc.)
answers     — list of returned IP addresses
TTL         — time-to-live of the DNS record
rejected    — whether DNS server refused the query
```

notice.log

Zeek's own alert system. When Zeek's built-in scripts detect something suspicious (port scan, weird behavior), it writes here. Format:

```
note        — alert type (e.g. Scan::Port_Scan)
msg         — human-readable message
src         — source IP that triggered the notice
```

Other logs

- `ssl.log` — TLS handshake details, certificate info, cipher suites
- `files.log` — files transferred over HTTP/FTP/SMTP
- `ssh.log` — SSH connection attempts
- `software.log` — detected software versions (servers, clients)
- `weird.log` — protocol violations and anomalies Zeek couldn't categorize

---

Zeek Commands You Need to Know

```
# Start everything (deploy = stop + install + start)
sudo /opt/zeek/bin/zeekctl deploy

# Check running status
sudo /opt/zeek/bin/zeekctl status

# Stop Zeek
sudo /opt/zeek/bin/zeekctl stop

# Start Zeek (if already deployed/configured)
sudo /opt/zeek/bin/zeekctl start

# Watch live conn.log as traffic comes in
sudo tail -f /opt/zeek/logs/current/conn.log

# Watch live http.log
sudo tail -f /opt/zeek/logs/current/http.log

# Filter conn.log for connections from Dell (attacker)
sudo grep "192.168.10.2" /opt/zeek/logs/current/conn.log

# Filter for S0 state (attack signature)
sudo grep "S0" /opt/zeek/logs/current/conn.log

# Count S0 connections
sudo grep -c "S0" /opt/zeek/logs/current/conn.log

# Filter http.log for Selenium user-agent
sudo grep "Mozilla" /opt/zeek/logs/current/http.log
```

```
# Filter for 408 responses (Slowloris timeouts)
sudo grep "408" /opt/zeek/logs/current/http.log

# Check log directory
sudo ls -lh /opt/zeek/logs/current/
```

Configuration file

`/opt/zeek/etc/node.cfg` — defines which interface Zeek monitors:

```
[zeek]
type=standalone
host=localhost
interface=eth0    ← this is what you set
```

Where logs rotate

Zeek rotates logs hourly into timestamped directories:

```
/opt/zeek/logs/2026-06-23/
/opt/zeek/logs/2026-06-23/conn.23:00:00-00:00:00.log.gz
```

Reading Zeek Logs with Python (for Phase 2)

```
import pandas as pd

# Zeek conn.log is tab-separated with a header starting with #
# The column names are on the line starting with #fields
df = pd.read_csv(
    '/opt/zeek/logs/current/conn.log',
    sep='\t',
    comment='#',
    names=[
        'ts', 'uid', 'id.orig_h', 'id.orig_p',
        'id.resp_h', 'id.resp_p', 'proto', 'service',
        'duration', 'orig_bytes', 'resp_bytes', 'conn_state',
        'local_orig', 'local_resp', 'missed_bytes', 'history',
        'orig_pkts', 'orig_ip_bytes', 'resp_pkts', 'resp_ip_bytes',
'tunnel_parents'
    ],
    na_values='-'  # Zeek uses '-' for missing values
)

# Convert timestamp to datetime
df['ts'] = pd.to_datetime(df['ts'], unit='s')

# Filter to only TCP traffic
tcp = df[df['proto'] == 'tcp']

# Find S0 connections (Slowloris signature)
slowloris_candidates = df[df['conn_state'] == 'S0']

# Find connections from Selenium bot
selenium = df[df['id.orig_h'] == '192.168.10.3']

print(df.head())
print(df['conn_state'].value_counts())
```

PART 2 — RITA (Real Intelligence Threat Analytics)

What is RITA?

RITA is a **post-analysis tool** that takes Zeek logs as input and applies threat hunting analytics on top of them. Where Zeek generates raw data, RITA interprets that data to find patterns indicating malicious behavior.

RITA v5 runs entirely in **Docker** — it spins up three containers:

1. **rita** — the main RITA binary
2. **clickhouse** — a column-oriented database that stores all the analysis results
3. **syslog-ng** — log receiver that ingests Zeek logs in real-time

---

RITA's Core Detection Capabilities

1. Beacon Detection (Most Important)

A **beacon** is a network connection that repeats at regular intervals — the classic behavior of malware calling back to its command-and-control (C2) server.

How RITA detects it:

- Collects all connections between two IP pairs over time
- Computes **inter-arrival time** between connections (time between connection N and N+1)
- Calculates statistical regularity: standard deviation, skewness, kurtosis
- Low standard deviation in inter-arrival times = highly regular = beacon

Example: Malware phones home every 300 seconds. RITA sees:

```
192.168.10.5 → evil.c2.com: connections at T=0, T=301, T=599, T=902...
Inter-arrival: 301, 298, 303... → std dev ≈ 2.5 → very regular → beacon score
HIGH
```

Why RITA misses Selenium bot: Selenium doesn't have a regular interval. It visits the page, waits 1-3 seconds (random), visits again. Inter-arrival times are: 1.2, 2.8, 1.1, 3.0, 1.7... The randomness makes RITA's beacon score very low. → Not flagged.

2. Long Connection Analysis

RITA flags connections that stay open for unusually long time:

- Normal web request: 10ms – 2 seconds
- File download: seconds to minutes

- C2 backdoor keeping channel open: hours

It calculates duration percentiles across all connections and flags statistical outliers.

3. DNS Tunneling Detection

DNS can be used to exfiltrate data by encoding it in DNS queries:

```
base64encodeddata.evil.com → A record query
```

RITA detects this by looking at:

- DNS query length distribution (tunneling queries are very long)
- Number of unique subdomains for a domain (tunneling generates many random subdomains)
- DNS data volume (unusually high bytes in DNS answers)

4. Rare Connections

Identifies IP pairs that communicated very few times but with unusual characteristics. Sometimes an attacker's initial foothold connection appears only once.

5. Threat Scoring

RITA aggregates all signals into a score per IP address. Format:

```
Source IP     Score   Connections  Beacons  Long Conns
192.168.10.2  0.87    1247         2        0
192.168.10.3  0.02    52           0        0     ← Selenium: low score
```

RITA Commands

```
# Check Docker containers are running
sudo docker ps

# Expected output:
# rita_rita_1        — main RITA process
# rita_clickhouse_1  — database
# rita_syslog-ng_1   — log ingestion

# Import Zeek logs into RITA for a specific dataset name
rita import --logs /opt/zeek/logs/current/ --dataset lab-session-01

# List available datasets
rita list

# Show threat scores for all IPs in a dataset
rita show-beacons lab-session-01

# Show long connections
rita show-long-connections lab-session-01

# Show DNS analysis
rita show-dns-fqdns lab-session-01

# Export results to HTML report
```

```
rita html-report lab-session-01 --output ./report/

# Delete a dataset
rita delete lab-session-01
```

ClickHouse — Why It Exists in RITA

ClickHouse is a **column-oriented** database (as opposed to row-oriented like MySQL/PostgreSQL).

Why it matters for log analysis:

Row-oriented (traditional): data stored as:

```
[row1: ts=1.0, ip=192.168.10.2, bytes=189, state=S0]
[row2: ts=1.1, ip=192.168.10.2, bytes=189, state=S0]
```

To aggregate: `SELECT AVG(bytes) FROM conn` → must scan every row, reading ALL columns even though you only want bytes.

Column-oriented (ClickHouse): data stored as:

```
[ts column:    1.0, 1.1, 1.2, 1.3, ...]
[ip column:    192.168.10.2, 192.168.10.2, ...]
[bytes column: 189, 189, 195, 189, ...]
[state column: S0, S0, SF, S0, ...]
```

To aggregate: `SELECT AVG(bytes)` → reads ONLY the bytes column. On millions of rows, this is 10-100x faster.

For RITA's use case (statistical analysis over millions of Zeek log entries), ClickHouse is essential. A regular DB would be too slow.

PART 3 — APACHE BENCH (ab)

What is Apache Bench?

Apache Bench (`ab`) is a **HTTP load testing and benchmarking tool** that ships with Apache HTTP Server. It sends a configurable number of HTTP requests as fast as possible and measures server response times.

In your project: used as a **volumetric attack tool** (HTTP flood).

How Apache Bench Works

1. Opens N concurrent TCP connections to the target
2. Sends HTTP GET requests over those connections as fast as possible

3. Waits for responses
4. Measures: requests per second, response time distribution, failure rate
5. Reports statistics when done

The attack effect: the target server must process each request — CPU usage spikes, memory fills, connection pool exhausts. Legitimate users get slow responses or timeouts.

Apache Bench Commands

```
# Basic: 1000 requests, 10 at a time
ab -n 1000 -c 10 http://192.168.10.1/

# Heavy: 10000 requests, 100 concurrent — realistic DoS test
ab -n 10000 -c 100 http://192.168.10.1/

# With custom headers
ab -n 1000 -c 50 -H "Accept-Encoding: gzip" http://192.168.10.1/

# POST request with data
ab -n 1000 -c 10 -m POST -p data.txt -T "application/json"
http://192.168.10.1/api/

# Time-based (run for 30 seconds instead of fixed count)
ab -t 30 -c 50 http://192.168.10.1/

# Keep-alive connections (more efficient attack, harder to detect)
ab -n 5000 -c 100 -k http://192.168.10.1/
```

Key flags:

- `-n` = total number of requests
- `-c` = number of concurrent connections (parallel requests)
- `-t` = time limit in seconds (instead of -n)
- `-k` = use HTTP KeepAlive (reuse connections, more efficient)
- `-H` = add a custom HTTP header
- `-m` = HTTP method (GET/POST/PUT/DELETE)

What ab Looks Like in Zeek

In `conn.log`:

- High volume of `SF` connections (successful, but many per second)
- All from `192.168.10.2` to `192.168.10.1:80`
- Short duration (milliseconds each)
- `orig_bytes` consistent (same GET request each time)

- `resp_bytes` consistent (same HTML page each time)

In `http.log`:

- Hundreds/thousands of `GET / HTTP/1.0` or `HTTP/1.1` entries
- All returning `200` (if server is handling them) or `503` (if overwhelmed)
- User-Agent: `ApacheBench/2.3` — very easy to identify!

This is why ab is easily detected: the User-Agent gives it away, and the volume is obviously not human.

PART 4 — SLOWLORIS

What is Slowloris?

Slowloris is a **low-and-slow DoS attack tool**. Unlike volumetric attacks (Apache Bench), it uses very few connections but keeps them alive indefinitely, exhausting the server's connection pool.

The name comes from the slow loris animal — it moves very slowly.

How Slowloris Works

Normal HTTP request:

```
Client → SYN
Server → SYN-ACK
Client → ACK
Client → "GET / HTTP/1.1\r\nHost: 192.168.10.1\r\n\r\n"   ← complete request
Server → "HTTP/1.1 200 OK\r\n...<html>..."                  ← response
Client → FIN
```

Slowloris attack:

```
Client → SYN
Server → SYN-ACK
Client → ACK
Client → "GET / HTTP/1.1\r\nHost: 192.168.10.1\r\n"       ← starts header
Client → (waits 10 seconds, sends one more header line)
Client → "X-Custom-Header: value\r\n"                       ← still not complete
Client → (waits 10 more seconds...)
Client → "X-Another: value\r\n"                             ← never sends final
\r\n
```

The server waits patiently for the complete request. Meanwhile, that connection slot is occupied. Slowloris opens hundreds of these. When the server's connection limit is hit (default Nginx: 1024), legitimate users get `503 Service Unavailable`.

Slowloris Commands (Python, Dell)

```
# Basic: attack port 80, 150 sockets
slowloris 192.168.10.1

# Specify port and socket count
slowloris 192.168.10.1 -p 80 -s 500

# HTTPS attack
slowloris 192.168.10.1 -p 443 --ssl

# Specify sleep time between header sends (default: 15 sec)
slowloris 192.168.10.1 --sleeptime 10

# Verbose output
slowloris 192.168.10.1 -v

# UDP mode (different attack vector)
slowloris 192.168.10.1 --udp
```

Key flags:

- `-p` = target port (default: 80)
- `-s` = number of sockets to open (default: 150)
- `--ssl` = use HTTPS (port 443)
- `--sleeptime` = seconds between keep-alive header sends
- `-v` = verbose, shows each socket's status

What Slowloris Looks Like in Zeek

In `conn.log`:

```
ts         uid    orig_h         orig_p  resp_h        resp_p  proto  service
duration   orig_b  resp_b  state
1782...    Cxxx   192.168.10.2  54309   192.168.10.1  80      tcp    http
0.014      189     0       S0
1782...    Cyyy   192.168.10.2  54310   192.168.10.1  80      tcp    http
0.014      189     0       S0
1782...    Czzz   192.168.10.2  54311   192.168.10.1  80      tcp    http
0.015      189     0       S0
```

Pattern:

- `state = S0` — SYN sent but connection never fully established from Zeek's perspective
- `resp_bytes = 0` — server never sent a response
- `orig_bytes ≈ 189` — the partial HTTP header Slowloris sends
- Many connections from same source IP simultaneously

In `http.log`:

- `408 Request Timeout` — server timed out waiting for complete headers

- No User-Agent (or partial one)

Why Slowloris is detectable: The `S0` state + zero `resp_bytes` + many simultaneous connections from one IP is a very clear statistical signature. Even without RITA, a simple threshold rule catches it.

PART 5 — LOCUST

What is Locust?

Locust is a **Python-based load testing framework**. It simulates many concurrent users sending HTTP requests to a target. Unlike Apache Bench, Locust gives you full programmatic control over user behavior — think/wait times, request sequences, weighted task selection.

In your project: used as **benign traffic** (trafic bénin) — simulates realistic user load.

How Locust Works

You define user behavior as Python classes:

```
from locust import HttpUser, task, between

class WebUser(HttpUser):
    # User waits 1-3 seconds between requests (realistic think time)
    wait_time = between(1, 3)

    @task(3)   # weight=3: called 3x more often than weight=1 tasks
    def visit_home(self):
        self.client.get("/")

    @task(1)
    def visit_about(self):
        self.client.get("/about")
```

Locust spawns multiple instances of this class, each running as a greenlet (lightweight coroutine). They all share a single event loop, making it efficient for simulating thousands of concurrent users with minimal CPU.

Locust Architecture

```
Locust process
├── Master (orchestrates)
└── Workers (each runs many Users)
    ├── User instance 1 → GET / → wait 2s → GET /about → wait 1s → ...
    ├── User instance 2 → GET / → wait 3s → GET / → wait 2s → ...
    └── User instance N → ...
```

In headless mode (your usage), master and worker are the same process.

Locust Commands

```
# Headless run: 10 users, spawn 2/second, run 30 seconds
locust -f ~/locustfile.py --host=http://192.168.10.1 --headless -u 10 -r 2 -t
30s

# Web UI mode (opens browser dashboard at localhost:8089)
locust -f ~/locustfile.py --host=http://192.168.10.1

# Run longer with more users
locust -f ~/locustfile.py --host=http://192.168.10.1 --headless -u 50 -r 5 -t 5m

# Output CSV statistics
locust -f ~/locustfile.py --host=http://192.168.10.1 --headless -u 10 -r 2 -t
60s --csv=results

# With custom wait time override
locust -f ~/locustfile.py --host=http://192.168.10.1 --headless -u 100 -r 10 -t
2m
```

Key flags:

- `-f` = path to locustfile
- `--host` = target base URL
- `--headless` = no web UI, run immediately
- `-u` = number of users to spawn total
- `-r` = spawn rate (users per second)
- `-t` = total run time (30s, 5m, 1h)
- `--csv` = output statistics to CSV files

Locust Output Explained

```
Type   Name    # reqs  # fails  Avg  Min  Max  Med  req/s  failures/s
GET    /       108     4(3.7%)  3    2    7    3    3.65   0.14
GET    /about  33      33(100%) 3    1    4    3    1.12   1.12

# reqs    — total requests made
# fails   — requests that got non-2xx response
Avg       — average response time (milliseconds)
req/s     — requests per second (throughput)
```

The `/about` 100% failure is because Nginx doesn't have that page — returns 404. Normal for your setup.

What Locust Looks Like in Zeek

In `conn.log`:

- Multiple `SF` connections from `192.168.10.3` (MacBook)
- Regular timing pattern (every 1-3 seconds, one connection)
- `service = http`
- Normal byte counts (small request, medium HTML response)

In `http.log`:

- `GET /` requests
- User-Agent: `python-httpx/...` or `python-requests/...` (Locust's default HTTP client)
- Status 200

Key observation: Locust's User-Agent reveals it's a Python client — not a real browser. RITA partially detects it because the timing is slightly regular. This is why Locust is "⚠ Partial" detection in your matrix.

PART 6 — SELENIUM

What is Selenium?

Selenium is a **browser automation framework** originally built for web application testing. It controls a real browser (Chrome, Firefox, Safari) programmatically. From the network's perspective, traffic generated by Selenium is **completely identical to human-generated traffic**.

This is the core of your research challenge.

How Selenium Works Internally

```
Java Code
    ↓
Selenium WebDriver API
    ↓
ChromeDriver (Google's official driver binary)
    ↓
Chrome DevTools Protocol (CDP)
    ↓
Actual Chrome Browser Process (headless)
    ↓
Real TCP/IP connection to 192.168.10.1:80
    ↓
Real HTTP request with full browser fingerprint
```

Every layer is real:

- **Real TLS handshake** — Chrome negotiates TLS exactly as it would for a human
- **Real User-Agent** — `Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)`

`AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36`

- **Real HTTP headers** — Accept, Accept-Language, Accept-Encoding, Cache-Control, etc.
- **Real TCP behavior** — same congestion control, same window sizes as a human session

---

ChromeDriver and WebDriver Protocol

ChromeDriver is a separate binary that acts as a bridge:

- Receives JSON commands over HTTP from your Java code
- Translates them to Chrome DevTools Protocol commands
- Sends them to the Chrome browser instance
- Returns results back to your Java code

When you do `driver.get("http://192.168.10.1")`:

1. Java sends JSON `{"method": "Page.navigate", "params": {"url": "http://192.168.10.1"}}` to ChromeDriver
2. ChromeDriver forwards to Chrome via CDP
3. Chrome opens a real TCP socket, does a real HTTP GET, renders the page
4. Returns the loaded page state back through the chain

Your Maven Project Structure

```
~/selenium-bot/selenium-bot/
├── pom.xml                          ← Maven build file (dependencies)
└── src/
    └── main/
        └── java/
            └── com/
                └── ids/
                    └── bot/
                        └── App.java ← Your main bot code
```

pom.xml — Dependencies

```
<dependencies>
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.21.0</version>
    </dependency>
</dependencies>
```

This pulls in the full Selenium Java library, which includes:

- `selenium-chrome-driver` — ChromeDriver management

- `selenium-remote-driver` — WebDriver protocol implementation
- `selenium-support` — helper utilities

Current App.java (what you have)

```
package com.ids.bot;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

public class App {
    public static void main(String[] args) throws InterruptedException {
        ChromeOptions options = new ChromeOptions();
        options.addArguments("--headless");          // no GUI window
        options.addArguments("--no-sandbox");        // required in some
environments
        options.addArguments("--disable-dev-shm-usage"); // shared memory issue
fix

        WebDriver driver = new ChromeDriver(options);

        String target = "http://192.168.10.1";

        for (int i = 0; i < 50; i++) {
            driver.get(target);                      // real HTTP GET
            System.out.println("Visit " + (i+1) + ": " + driver.getTitle());
            // random sleep between 1000 and 3000 milliseconds
            Thread.sleep(1000 + (long)(Math.random() * 2000));
        }

        driver.quit();  // closes browser, kills ChromeDriver
        System.out.println("Bot completed.");
    }
}
```

Selenium Java API — Useful Methods for Phase 2

```
// Navigation
driver.get("http://192.168.10.1");           // navigate to URL
driver.navigate().back();                      // browser back
driver.navigate().refresh();                   // F5 refresh
driver.getCurrentUrl();                        // get current URL
driver.getTitle();                             // get page title

// Finding elements
driver.findElement(By.id("search-box"));      // by HTML id
driver.findElement(By.cssSelector(".nav a")); // by CSS selector
driver.findElement(By.xpath("//h1"));         // by XPath

// Interacting
element.click();
element.sendKeys("search term");
element.getText();

// Waiting (important for dynamic pages)
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.presenceOfElementLocated(By.id("result")));
```

```
// JavaScript execution
((JavascriptExecutor) driver).executeScript("window.scrollTo(0,
document.body.scrollHeight)");

// Cookies (makes traffic look even more realistic)
driver.manage().getCookies();
driver.manage().addCookie(new Cookie("session", "abc123"));
```

Running Selenium

```
cd ~/selenium-bot/selenium-bot

# Compile
mvn compile

# Run the bot
mvn exec:java -Dexec.mainClass="com.ids.bot.App"

# Compile and run in one command
mvn compile exec:java -Dexec.mainClass="com.ids.bot.App"

# Run with custom JVM args
mvn exec:java -Dexec.mainClass="com.ids.bot.App" -Dexec.args="arg1 arg2"
```

What you see in terminal:

```
Visit 1: Welcome to nginx!
Visit 2: Welcome to nginx!
...
Visit 50: Welcome to nginx!
Bot completed.
```

What Selenium Looks Like in Zeek

In `http.log`:

```
192.168.10.3  GET  /  200  Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36
```

Compare to Apache Bench:

```
192.168.10.2  GET  /  200  ApacheBench/2.3
```

And Locust:

```
192.168.10.3  GET  /  200  python-httpx/0.27.0
```

The User-Agent is the biggest tell — but a sophisticated attacker could spoof it. And RITA sees no beacon pattern, no anomaly. This is exactly the detection gap your Autoencoder must fill.

PART 7 — AUTOENCODER

What is an Autoencoder?

An Autoencoder is a **neural network trained to compress data and then reconstruct it**. The key insight: it's trained only on normal data, so it learns to reconstruct normal patterns well. When it sees anomalous data, it reconstructs it poorly — the reconstruction error becomes the anomaly signal.

This is **unsupervised anomaly detection** — no labeled attack samples needed for training.

Architecture — Your Specific Design

```
Input (N features)
    ↓
Dense(32, activation='relu')    ← Encoder layer 1: compress to 32 dimensions
    ↓
Dense(16, activation='relu')    ← Encoder layer 2: compress to 16 dimensions
    ↓
Dense(8, activation='relu')     ← Bottleneck: most compressed representation (8
dims)
    ↓
Dense(16, activation='relu')    ← Decoder layer 1: expand back to 16
    ↓
Dense(32, activation='relu')    ← Decoder layer 2: expand back to 32
    ↓
Dense(N, activation='sigmoid')  ← Output: reconstruct original N features
```

The **bottleneck** (8 dimensions) forces the network to learn a compressed representation. It can only keep the most statistically important patterns. Normal traffic patterns are learned and compressed efficiently. Abnormal traffic doesn't fit the compression scheme → higher reconstruction error.

Why Sigmoid on Output?

After MinMax scaling, all input features are in [0, 1]. Sigmoid output is also [0, 1]. This means the reconstruction is directly comparable to the input, and MSE loss makes sense.

If you use StandardScaler instead, use `activation='linear'` on the output (unbounded values).

Training Logic (Phase 2 Code)

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow import keras

# ─── 1. Load and prepare data ──────────────────────────────────
```

```
df = pd.read_csv('/opt/zeek/logs/current/conn.log',
    sep='\t', comment='#',
    names=['ts', 'uid', 'id.orig_h', 'id.orig_p', 'id.resp_h', 'id.resp_p',
           'proto', 'service', 'duration', 'orig_bytes', 'resp_bytes',
           'conn_state', 'local_orig', 'local_resp', 'missed_bytes', 'history',
           'orig_pkts', 'orig_ip_bytes', 'resp_pkts', 'resp_ip_bytes',
'tunnel_parents'],
    na_values='-')

# ── 2. Feature engineering ──────────────────────────────────

# Fill missing numeric values
df['duration'] = df['duration'].fillna(0)
df['orig_bytes'] = df['orig_bytes'].fillna(0)
df['resp_bytes'] = df['resp_bytes'].fillna(0)
df['orig_pkts'] = df['orig_pkts'].fillna(0)
df['resp_pkts'] = df['resp_pkts'].fillna(0)

# Encode conn_state as integer
state_map = {'S0': 0, 'SF': 1, 'REJ': 2, 'S1': 3, 'S2': 4,
             'S3': 5, 'RSTO': 6, 'RSTR': 7, 'OTH': 8}
df['conn_state_enc'] = df['conn_state'].map(state_map).fillna(8)

# Encode protocol
proto_map = {'tcp': 0, 'udp': 1, 'icmp': 2}
df['proto_enc'] = df['proto'].map(proto_map).fillna(0)

# Select features for ML
features = ['duration', 'orig_bytes', 'resp_bytes', 'orig_pkts',
            'resp_pkts', 'conn_state_enc', 'proto_enc']

X = df[features].values

# ── 3. Split: normal traffic only for training ──────────────

# Label by source IP (or however you've tagged traffic)
normal_mask = df['id.orig_h'] == '192.168.10.3'  # MacBook = benign
X_normal = X[normal_mask]
X_anomaly = X[~normal_mask]  # everything else for evaluation

X_train, X_val = train_test_split(X_normal, test_size=0.2, random_state=42)

# ── 4. Scale features ───────────────────────────────────────

scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)   # fit ONLY on training data
X_val_scaled = scaler.transform(X_val)            # transform with same scaler
X_anomaly_scaled = scaler.transform(X_anomaly)

# ── 5. Build Autoencoder ────────────────────────────────────

n_features = X_train_scaled.shape[1]

autoencoder = keras.Sequential([
    keras.layers.Dense(32, activation='relu', input_shape=(n_features,)),
    keras.layers.Dense(16, activation='relu'),
    keras.layers.Dense(8, activation='relu'),      # bottleneck
    keras.layers.Dense(16, activation='relu'),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(n_features, activation='sigmoid')
])

autoencoder.compile(optimizer='adam', loss='mse')
```

```
autoencoder.summary()

# ─── 6. Train ──────────────────────────────────────────────────

history = autoencoder.fit(
    X_train_scaled, X_train_scaled,   # input = output (reconstruction task)
    epochs=50,
    batch_size=32,
    validation_data=(X_val_scaled, X_val_scaled),
    shuffle=True
)

# ─── 7. Set anomaly threshold ──────────────────────────────────

# Compute reconstruction error on validation set (normal traffic)
val_reconstructed = autoencoder.predict(X_val_scaled)
val_mse = np.mean(np.power(X_val_scaled - val_reconstructed, 2), axis=1)

threshold = np.mean(val_mse) + 2 * np.std(val_mse)
print(f"Anomaly threshold: {threshold:.6f}")

# ─── 8. Detect anomalies ───────────────────────────────────────

# Test on all traffic
X_all_scaled = scaler.transform(X)
reconstructed = autoencoder.predict(X_all_scaled)
mse = np.mean(np.power(X_all_scaled - reconstructed, 2), axis=1)

df['reconstruction_error'] = mse
df['is_anomaly'] = mse > threshold

# Results
print(df.groupby('id.orig_h')['is_anomaly'].mean())
print(f"\nAnomaly rate: {df['is_anomaly'].mean():.2%}")
```

Key Hyperparameters to Tune

Parameter	**Value**	**Effect**
Bottleneck size (8)	Smaller = more compression	Too small → high error even for normal; too large → anomalies not detected
Epochs (50)	More = better fit	Too many → overfitting to normal; too few → underfitted
Threshold (mean + 2σ)	Higher = fewer false positives	Lower → more sensitive but more false alarms
Batch size (32)	Affects training stability	Larger batches = more stable gradients
Learning rate (Adam default: 0.001)	Smaller = slower but stable	

Evaluation Metrics (Phase 2)

```
from sklearn.metrics import classification_report, roc_auc_score, roc_curve
import matplotlib.pyplot as plt

# Assuming you have true labels
y_true = (df['id.orig_h'] == '192.168.10.2').astype(int)  # Dell = attack = 1
y_pred = df['is_anomaly'].astype(int)
y_score = df['reconstruction_error']

print(classification_report(y_true, y_pred, target_names=['Normal', 'Anomaly']))

# ROC AUC
auc = roc_auc_score(y_true, y_score)
print(f"ROC AUC: {auc:.4f}")

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_true, y_score)
plt.plot(fpr, tpr, label=f'AUC = {auc:.3f}')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate (Detection Rate)')
plt.title('Autoencoder ROC Curve')
plt.legend()
plt.savefig('roc_curve.png')
```

PART 8 — NGINX (Victim Web Server)

What is Nginx Here?

Nginx is the **target** — it runs on the Raspberry Pi and serves as the web server that receives all traffic (attacks and benign). Without a running web server, there's nothing to attack and no HTTP logs in Zeek.

Key Nginx Commands

```
# Start nginx
sudo systemctl start nginx

# Enable at boot
sudo systemctl enable nginx

# Check status
sudo systemctl status nginx

# Stop
sudo systemctl stop nginx

# Reload config without restarting (for config changes)
sudo systemctl reload nginx

# Test config syntax before applying
sudo nginx -t
```

```
# View access log (who's hitting the server)
sudo tail -f /var/log/nginx/access.log

# View error log
sudo tail -f /var/log/nginx/error.log
```

Nginx Access Log Format

```
192.168.10.2 - - [23/Jun/2026:14:04:27 +0300] "GET /?560 HTTP/1.1" 408 0 "-"
"Mozilla/5.0..."
```

Fields: `IP - - [timestamp] "METHOD URI PROTOCOL" STATUS BYTES "REFERER" "USER-AGENT"`

Status 408 = Request Timeout (Slowloris signature in Nginx logs).

PART 9 — DOCKER (for RITA)

Why Docker for RITA?

RITA v5 requires ClickHouse (complex C++ application) and syslog-ng. Running these natively would require complex installation and configuration. Docker packages everything into containers that run in isolation.

Docker Commands You Need

```
# Check running containers
sudo docker ps

# Expected output when RITA is running:
# CONTAINER ID  IMAGE                        STATUS   PORTS   NAMES
# abc123        ghcr.io/activecm/rita:...    Up 5m            rita_rita_1
# def456        clickhouse/clickhouse-...    Up 5m            rita_clickhouse_1
# ghi789        lscr.io/linuxserver/...      Up 5m            rita_syslog-ng_1

# View logs of a container
sudo docker logs rita_rita_1
sudo docker logs rita_clickhouse_1

# Stop all RITA containers
sudo docker compose -f /opt/rita/docker-compose.yml down

# Start all RITA containers
sudo docker compose -f /opt/rita/docker-compose.yml up -d

# Restart
sudo docker compose -f /opt/rita/docker-compose.yml restart

# Check disk usage by Docker
sudo docker system df

# Remove stopped containers and unused images (free disk space)
```

```
sudo docker system prune
```

QUICK REFERENCE — All Commands

Raspberry Pi (192.168.10.1)

```
# Network
ip addr show eth0               # Check lab IP
ip addr show wlan0              # Check WiFi IP
ping 192.168.10.2               # Ping Dell
ping 192.168.10.3               # Ping MacBook

# Zeek
sudo /opt/zeek/bin/zeekctl deploy    # Start fresh
sudo /opt/zeek/bin/zeekctl status    # Check running
sudo /opt/zeek/bin/zeekctl stop      # Stop
sudo tail -f /opt/zeek/logs/current/conn.log    # Live conn log
sudo tail -f /opt/zeek/logs/current/http.log    # Live http log
sudo grep "S0" /opt/zeek/logs/current/conn.log | wc -l   # Count S0s
sudo grep "192.168.10.2" /opt/zeek/logs/current/http.log # Dell traffic

# Nginx
sudo systemctl status nginx
sudo systemctl start nginx
sudo tail -f /var/log/nginx/access.log

# Docker / RITA
sudo docker ps
sudo docker logs rita_rita_1
```

Dell (192.168.10.2)

```
rem Network
ipconfig
ping 192.168.10.1
ping 192.168.10.3

rem Apache Bench
ab -n 1000 -c 50 http://192.168.10.1/
ab -n 10000 -c 100 -k http://192.168.10.1/

rem Slowloris
slowloris 192.168.10.1 -p 80 -s 500
slowloris 192.168.10.1 -p 80 -s 200 -v
```

MacBook (192.168.10.3)

```
# Network
ifconfig en7                    # Lab ethernet
ifconfig en0                    # WiFi
ping -c 4 192.168.10.1
ping -c 4 192.168.10.2

# Locust
locust -f ~/locustfile.py --host=http://192.168.10.1 --headless -u 10 -r 2 -t
```

```
30s
locust -f ~/locustfile.py --host=http://192.168.10.1 --headless -u 50 -r 5 -t 2m

# Selenium
cd ~/selenium-bot/selenium-bot
mvn exec:java -Dexec.mainClass="com.ids.bot.App"
mvn compile exec:java -Dexec.mainClass="com.ids.bot.App"
```

Last updated: 23 June 2026 — Phase 1 complete